

Agenda

- Comparable
- Comparator
- Collection Framework
- FailSafe and FailFast Iterator
- Traversal
- ~~List~~

Comparable<>

- Standard for comparing the current object to the other object.
- Has single abstract method `int compareTo(T other);`
- In `java.lang` package.
- Used by various methods like `Arrays.sort(Object[])`, ...
- It does the comparison for the natural ordering

```
public class Product implements Comparable<Product> {
    private int id;
    private String name;
    private double price;

    public Product() {
    }

    public Product(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    @Override
    public String toString() {
        return "Product [id=" + id + ", name=" + name + ", price=" + price + "]";
    }

    @Override
    public int compareTo(Product other) {
        int diff = this.id - other.id;
        return diff;
    }

    public static void main(String[] args) {
        Product[] arr1 = {
            new Product(4, "Pen", 20.0),
            new Product(3, "Pencil", 8.0),
            new Product(1, "Erasure", 5.0),
            new Product(5, "Notebook", 50.0),
        }
    }
}
```

```
};

System.out.println("Before Sort: ");
for (Product p : arr1)
    System.out.println(p.toString());

Arrays.sort(arr1);

System.out.println("\nAfter Sort: ");
for (Product p : arr1)
    System.out.println(p.toString());
}
```

Comparator<>

- Standard for comparing two (other) objects.
- Has single abstract method `int compare(T obj1, T obj2);`
- In `java.util` package.
- Used by various methods like `Arrays.sort(T[], comparator), ...`

```
// pre-defined interface
interface Comparator<T> {
    int compare(T obj1, T obj2);
}
```

```
public static void main(String[] args) {
    Product[] arr = {
        new Product(4, "Pen", 20.0, "Sec Stationary"),
        new Product(3, "Pencil", 8.0, "Pri Stationary"),
        new Product(1, "Erasure", 5.0, "Pri Stationary"),
        new Product(5, "Notebook", 50.0, "Common Stationary"),
    };

    System.out.println("Before Sort: ");
    for (Product p : arr)
        System.out.println(p.toString());

    class ProductNameComparator implements Comparator<Product>{
        @Override
        public int compare(Product p1, Product p2) {
            int diff = p1.getName().compareTo(p2.getName());
            return diff;
        }
    }

    Comparator<Product> productNameComparator = new ProductNameComparator();
    Arrays.sort(arr, productNameComparator);

    System.out.println("\nAfter Sort (by Name): ");
    for (Product p : arr)
```

```

        System.out.println(p.toString());

        class ProductPriceDescComparator implements Comparator<Product> {
            @Override
            public int compare(Product p1, Product p2) {
                if (p1.getPrice() > p2.getPrice())
                    return -1;
                if (p1.getPrice() < p2.getPrice())
                    return +1;
                return 0;
            }
        }

        Arrays.sort(arr, new ProductPriceDescComparator());

        System.out.println("\nAfter Sort (by Price Desc): ");
        for (Product p : arr)
            System.out.println(p.toString());
    }

```

- We can perform multilevel sorting as well

```

// Multi-level sorting
// -- 1st level: department,
// 2nd level: designation,
// 3rd level: salary(int)

class CustomComparator implements Comparator<Employee> {
    public int compare(Employee e1, Employee e2) {
        int diff = e1.getDepartment().compareTo(e2.getDepartment());
        if(diff == 0)
            diff = e1.getDesignation().compareTo(e2.getDesignation());
        if(diff == 0)
            diff = e1.getSalary() - e2.getSalary();
        return diff;
    }
}

Employee[] arr = { ... };
Arrays.sort(arr, new CustomComparator());

```

Collection Framework

- Collection framework is Library of reusable data structure classes that is used to develop application.
- Main purpose of collection framework is to manage data/objects in RAM efficiently.
- Collection framework was introduced in Java 1.2 and type-safe implementation is provided in 5.0 (using generics).
- Collection is available in java.util package.
- Java collection framework provides

1. Interfaces -- defines standard methods for the collections.
2. Implementations -- classes that implements various data structures.
3. Algorithms -- helper methods like searching, sorting, ...

Collection Hierarchy

- Interfaces: Iterable, Collection, List, Queue, Set, Map, Deque, SortedSet, SortedMap, ...
- Implementations: ArrayList, LinkedList, HashSet, HashMap, ...
- Algorithms: sort(), reverse(), max(), min(), ... -> in Collections class static methods

Collection interface

- Root interface in collection framework interface hierarchy.
- Most of collection classes are inherited from this interface (indirectly).
- Provides most basic/general functionality for any collection
- Abstract methods
 - boolean add(E e)
 - int size()
 - boolean isEmpty()
 - void clear()
 - boolean contains(Object o)
 - boolean remove(Object o)
 - boolean addAll(Collection<? extends E> c)
 - boolean containsAll(Collection<?> c)
 - boolean removeAll(Collection<?> c)
 - boolean retainAll(Collection<?> c)
 - Object[] toArray()
 - Iterator iterator() -- inherited from Iterable
- Default methods
 - default Stream stream()
 - default Stream parallelStream()
 - default boolean removeIf(Predicate<? super E> filter)

Iterable interface

- To traverse any collection it provides an Iterator.
- Enable use of for-each loop.
- In java.lang package
- Iterable yields an iterator
- Methods
 - Iterator iterator()
 - default Spliterator spliterator()
 - default void forEach(Consumer<? super T> action)

Iterator

- Part of collection framework (1.2)
- Methods

- boolean hasNext()
- E next()
- void remove()

Enumeration

Since Java 1.0

- Methods
 - boolean hasMoreElements()
 - E nextElement() (Demo02)

Collections class

- Helper/utility class that provides several static helper methods
- Methods
 - List reverse(List list);
 - List shuffle(List list);
 - void sort(List list, Comparator cmp)
 - E max(Collection list, Comparator cmp);
 - E min(Collection list, Comparator cmp);
 - List synchronizedList(List list);

Collection vs Collections

1. Collection interface

- All methods are public and abstract. They implemented in sub-classes.
- Since all methods are non-static, must be called on object.

```
Collection<Integer> list = new ArrayList<>();  
//List<Integer> list = new ArrayList<>();  
//ArrayList<Integer> list = new ArrayList<>();  
list.remove(new Integer(12));
```

2. Collections class

- Helper class that contains all static methods.
- We never create object of "Collections" class.

```
Collections.methodName(...);
```

Fail-fast vs Fail-safe Iterator

- If state of collection is modified (add/remove operation other than iterator methods) i.e structural change while traversing a collection using iterator and iterator methods fails (with

ConcurrentModificationException), then iterator is said to be Fail-fast.

- The collections from java.util package have fail-fast iterators
- e.g. Iterators from ArrayList, LinkedList, Vector, ...
- If iterator allows to modify the underlying collection (add/remove operation other than iterator methods) while traversing a collection (NO ConcurrentModificationException), then iterator is said to be Fail-safe.
- The collections from java.util.concurrent package have fail-safe iterators.
- e.g. Iterators from CopyOnWriteArrayList, ...
- If any changes are done in the collection using these iterators then the changes may not be reflected using the same iterator however by creating the new iterator we can get the changes displayed.

Traversal

1. Using Iterator

```
Iterator<Integer> itr = list.iterator();
while(itr.hasNext()) {
    Integer i = itr.next();
    System.out.println(i);
}
```

2. Using for-each loop

```
for(Integer i:list)
    System.out.println(i);

// gets converted into Iterator traversal internally

for(Iterator<Integer> itr = list.iterator(); itr.hasNext();) {
    Integer i = itr.next();
    System.out.println(i);
}
```

3. using for loop

```
for(int index=0; index<list.size(); index++) {
    Integer i = list.get(index);
    System.out.println(i);
}
```

4. Enumeration -- Traversing Vector (Java 1.0)

```
// v is Vector<Integer>
Enumeration<Integer> e = v.elements();
while(e.hasMoreElements()) {
    Integer i = e.nextElement();
    System.out.println(i);
}
```

- Enumeration behaves similar to fail-safe iterator.
- Since Java 1.0
- Methods
 - boolean hasMoreElements()
 - E nextElement()
- only useful when traversing with vector

Assignments

- Q1. Create an Employee class with fields id,name and salary. Create an array of 5 employees randomly ordered and then perform the sorting based on
 1. empid in ascending
 2. name in ascending
 3. salaray in descending
- Q2. For the above class create an collection and test the methods from the collection interface